

LifeOS Agent

PRD + 技术架构 + MVP 开发清单

融合修仙世界观的伴随式个人成长智能体，通过 Memory、Skills 与 Agent Harness
构建可观测、可进化的成长闭环。

LifeOS Agent PRD + 技术架构 + MVP 开发清单

1. 项目定位

项目名称：LifeOS Agent

一句话定位：

LifeOS Agent 是一个融合修仙世界观的伴随式个人成长智能体，通过日常记录理解用户，通过长期 Memory 沉淀个人画像，通过 Skills 固化成长方法，并通过 Agent Harness 追踪每次建议、执行与反馈，让 Agent 在陪伴用户成长的过程中持续自进化。

核心关键词：

- LifeOS
- 伴随式 Agent
- 个人成长轨迹
- 修仙世界观
- Memory
- Agent Skills
- Agent Harness
- 自进化
- 可视化体验
- 3D / 沉浸式成长空间

2. 用户痛点

目标用户是长期独自学习、做项目、准备比赛、探索方向的学生或个人开发者。

他们的问题不是“没有工具”，而是：

1. 日常记录碎片化，无法沉淀成长期成长轨迹。
2. 计划经常失败，但失败原因没有被系统性总结。
3. 个人状态、学习节奏、拖延模式缺少长期观察。
4. 普通日程工具只管理任务，不理解人的成长过程。
5. AI 助手通常只响应单次问题，缺少长期陪伴、自我调整和持续进化能力。

LifeOS Agent 解决的是：

一个人长期往前走时，需要一个能记录他、理解他、提醒他、复盘他，并和他一起进化的智能体。

3. 产品目标

LifeOS Agent 的目标不是做一个普通待办工具，而是构建一个“伴随式成长操作系统”。

核心目标：

- 1. 让用户用自然语言记录每天的学习、项目、状态和困惑。
- 2. Agent 自动生成修炼日志、复盘总结和下一步计划。
- 3. 系统持续更新用户长期 Memory。
- 4. Agent 根据用户状态选择合适 Skills。
- 5. Agent Harness 记录每次决策链路和反馈结果。
- 6. 系统根据反馈调整 Memory、Skill 参数和后续计划策略。
- 7. 将成长过程用修仙境界、心魔雷达图、功法熟练度、洞府看板、时间线长河等形式可视化。

4. 修仙世界观映射

LifeOS Agent 使用《凡人修仙传》式修为结构作为成长可视化框架，但不复刻原作剧情，只抽取适合个人成长系统的概念。

修仙概念	LifeOS 映射	产品含义
修士	用户本人	正在长期成长的人
洞府	个人成长工作台	首页、可视化空间、长期状态总览
修炼日志	日常记录	用户每天输入的学习、项目、情绪、困惑
修为境界	总成长阶段	用户整体成长水平
子境界	具体方向成长	技术栈、项目、习惯、长期目标的独立进度
功法	Agent Skills	可复用的成长方法和工作流
功法熟练度	Skill 效果评分	某个 Skill 对用户是否有效
心魔	阻碍模式	拖延、焦虑、熬夜、分心、逃避
瓶颈	阶段性卡点	连续失败、进展停滞、目标模糊
突破	成长跃迁	完成关键目标或形成稳定能力
灵根	用户画像	学习偏好、能力倾向、精力节律
法器	外部工具	日历、提醒、笔记、搜索、代码工具
阵法	多 Skill 编排	多个技能组合成的工作流
命运长河	成长时间线	长期记录、成果、低谷、突破的可视化轨迹

5. 境界系统设计

修为境界参考《凡人修仙传》结构：

练气期：一层至十三层

筑基期：初期 / 初期巅峰 / 中期 / 中期巅峰 / 后期 / 后期巅峰

结丹期：初期 / 初期巅峰 / 中期 / 中期巅峰 / 后期 / 后期巅峰

元婴期：初期 / 初期巅峰 / 中期 / 中期巅峰 / 后期 / 后期巅峰

化神期：初期 / 初期巅峰 / 中期 / 中期巅峰 / 后期 / 后期巅峰

成长映射：

境界	成长阶段
练气期	建立记录、复盘和执行习惯
筑基期	形成稳定学习节奏和项目推进能力
结丹期	能持续产出作品，形成阶段性成果
元婴期	形成个人方法论，Skills 可稳定复用
化神期	能进行长期规划、自我调整，并迁移能力到复杂目标

境界进度不只由任务完成数决定，而是综合计算：

境界进度 =
稳定执行
+ 复盘质量
+ 项目推进
+ 心魔改善
+ Skill 熟练度
+ 长期目标一致性

建议 MVP 权重：

稳定执行：25%
复盘质量：20%
项目推进：20%
心魔改善：15%
Skill 熟练度：10%
长期目标一致性：10%

6. 总境界与子境界

LifeOS Agent 区分 总境界 和 子境界。

总境界：

表示用户整体成长状态，由多个子境界综合计算。

例如：

总境界：练气六层

子境界：

表示用户在具体方向上的成长进度，可以绑定：

- 某条技术栈
- 某个项目

- 某个长期目标
- 某段修炼 session
- 某个生活习惯
- 某项能力维度

示例：

总境界：练气六层

子境界：

- AI Agent 技术栈：练气八层
- 算法能力：练气四层
- 项目实战：练气七层
- 英语表达：练气二层
- 作息与健康：练气三层

这个机制可以体现真实成长的不均衡性，避免一个总等级概括所有能力。

7. 核心功能

7.1 修炼日志输入

用户每天用自然语言输入：

今天学了 RAG，但是下午刷短视频浪费了两小时。
算法题没做完，有点焦虑。晚上看了一点 Agent Harness 的资料。

Agent 自动解析：

- 今日学习内容
- 今日成果
- 未完成事项
- 情绪状态
- 心魔类型
- 项目进展
- 明日建议
- 需要更新的 Memory
- 需要调用的 Skills

7.2 每日复盘

Agent 生成结构化修炼日志：

今日修炼：

- RAG 学习推进 40%
- Agent Harness 资料阅读完成 1 次
- 算法训练未完成

今日心魔：

- 下午分心
- 目标过大
- 焦虑感上升

明日建议：

- 上午安排 RAG 深度学习
- 下午安排轻任务：整理笔记
- 算法题减少为 1 道，降低启动阻力

7.3 阶段性规划

支持日 / 周 / 月计划。

规划不是静态生成，而是结合：

- 长期目标
- 当前境界
- 子境界进度
- 近期失败模式
- 用户精力节律
- 历史执行率

7.4 心魔识别

心魔是用户成长中的阻碍模式。

MVP 心魔类型：

- 拖延
- 分心
- 熬夜
- 焦虑
- 目标过大
- 缺少反馈
- 计划频繁失败

心魔不只是标签，而会影响后续计划生成。

例如：

如果用户连续多次下午分心：

系统更新 Memory：下午适合低认知任务

后续计划调整：上午安排深度任务，下午安排整理、复盘、轻量阅读

7.5 功法 Skills

功法是 Agent Skills 的产品化表达。

MVP Skills：

功法名	技术 Skill	功能
每日复盘诀	daily_reflection	生成每日复盘
周天规划术	planning	生成日/周/月计划
心魔照见法	obstacle_detection	识别阻碍模式
记忆凝练术	memory_consolidation	更新长期记忆
项目破阵诀	project_breakdown	拆解项目任务
知识炼化术	learning_summary	总结学习内容
闭关修炼法	deep_work_planning	生成专注学习计划

每个 Skill 至少包含：

```
{
  "name": "daily_reflection",
  "displayName": "每日复盘诀",
  "trigger": "用户提交每日记录",
  "input": ["daily_input", "retrieved_memory"],
  "output": ["reflection", "growth_delta", "next_plan"],
  "score": 0.82,
  "evolutionNotes": []
}
```

7.6 Agent Harness

Agent Harness 是 LifeOS Agent 的技术核心。

它不是普通日志系统，而是 Agent 自进化的训练场。

每次 Agent 行动都记录 trace：

用户输入
→ 检索了哪些 Memory
→ 选择了哪些 Skills
→ 生成了什么建议
→ 用户是否采纳
→ 执行结果如何
→ 哪些 Memory 被更新
→ 哪些 Skill 参数被调整
→ 下一轮策略如何变化

Harness 记录结构示例：

```
{
  "traceId": "trace_001",
  "input": "今天学了 RAG，但下午分心严重",
  "retrievedMemory": [
    "用户最近 3 次下午学习效率较低",
    "用户当前目标是完成 LifeOS Agent MVP"
  ],
  "selectedSkills": [
    "每日复盘诀",
    "心魔照见法",
    "周天规划术"
  ],
}
```

```

"agentOutput": {
  "reflection": "...",
  "nextPlan": "..."
},
"evaluation": {
  "planDifficulty": 0.72,
  "emotionalSupport": 0.81,
  "goalAlignment": 0.88
},
"memoryUpdates": [
  "下午适合轻量任务，不适合高强度学习"
],
"skillEvolution": [
  "planning.intensity 从 0.8 调整为 0.6"
]
}

```

7.7 Memory 系统

Memory 分为短期、中期、长期。

短期 Memory：
最近几天的记录、任务、状态

中期 Memory：
最近几周的执行模式、项目进展、反复心魔

长期 Memory：
用户目标、偏好、节律、能力画像、长期成长轨迹

Memory 更新原则：

1. 不保存所有原始聊天。
2. 从重复模式中提炼长期结论。
3. 支持更新、冲突修正和遗忘。
4. Memory 必须能解释 Agent 为什么这样建议。

Memory 示例：

```

{
  "type": "long_term_pattern",
  "content": "用户在上午更适合深度学习，下午容易分心，适合安排整理类任务。",
  "evidence": ["2026-06-24", "2026-06-26", "2026-06-28"],
  "confidence": 0.78,
  "lastUpdated": "2026-06-28"
}

```

8. 自进化机制

LifeOS Agent 的自进化不做成玄学，而是明确的工程闭环：

```

Daily Input
→ Memory Retrieval
→ Skill Selection
→ Reflection / Plan Generation
→ Harness Evaluation

```


- Memory Update
- Skill Evolution
- Next Strategy

自进化分三层：

8.1 Memory Evolution

从日常记录中提炼长期模式。

例如：

- 用户连续三次下午分心
- 更新长期 Memory：
- “下午适合低认知任务，上午适合深度学习”

8.2 Skill Evolution

根据用户反馈调整 Skill 参数。

例如：

- 用户反馈计划太重
- planning skill 的 intensity 从 0.8 降到 0.6
- 明日计划拆成更小任务

8.3 Planning Evolution

下一轮计划生成策略发生变化。

例如：

- 旧策略：每天安排 3 小时算法
- 新策略：每天 30 分钟算法 + 1 道题 + 晚间复盘

9. 可视化设计

视觉是 LifeOS Agent 的关键体验点。

MVP 页面建议 4 个：

9.1 洞府主页

核心首页。

展示：

- 当前总境界
- 今日状态
- 洞府 3D / 伪 3D 场景
- 今日修炼任务
- 当前心魔
- 近期突破

- 功法熟练度摘要

视觉建议：

- 中央是洞府 / 灵台 / 修炼场景
- 境界变化时有光效或灵气变化
- 不做普通 dashboard，要有沉浸感

9.2 修炼日志页

用户输入自然语言记录。

展示：

- 输入框
- Agent 解析结果
- 今日复盘
- 明日计划
- Memory 更新提示
- Skill 调用列表

9.3 成长轨迹页

展示长期成长。

包含：

- 命运长河时间线
- 关键节点
- 项目推进记录
- 境界突破记录
- 子境界变化

9.4 Harness 面板

展示技术核心。

包含：

- 本次输入
- 检索到的 Memory
- 调用的 Skills
- Agent 决策链路
- 评估结果
- Memory 更新

- Skill Evolution 变化

这个页面是极客组路演的关键，能证明项目不是普通成长 App。

10. 技术架构

10.1 前端

建议技术栈：

Next.js / React
Tailwind CSS
Framer Motion
Three.js / React Three Fiber
Recharts / ECharts
Zustand 或 Redux

模块：

Dashboard 洞府主页
Journal 修炼日志
Timeline 命运长河
Harness Trace 面板
Skill / Memory 可视化组件

10.2 后端

建议技术栈：

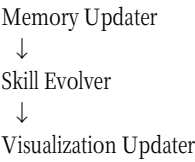
Node.js / Python
FastAPI 或 Express
SQLite / PostgreSQL
Vector Store 可选
LLM API

核心服务：

Agent Orchestrator
Memory Store
Skill Registry
Harness Trace Store
Evaluation Engine
Growth Engine
Visualization Data API

10.3 Agent 架构

User Input
↓
Input Parser
↓
Memory Retriever
↓
Skill Selector
↓
Agent Planner / Reflector
↓
Harness Evaluator
↓



10.4 数据模型

UserProfile

```
{
  "id": "user_001",
  "name": "修士",
  "overallRealm": "练气六层",
  "subRealms": [
    {
      "name": "AI Agent 技术栈",
      "realm": "练气八层",
      "progress": 0.68
    }
  ],
  "longTermGoals": [],
  "patterns": []
}
```

DailyLog

```
{
  "id": "log_001",
  "date": "2026-06-28",
  "rawInput": "...",
  "summary": "...",
  "achievements": [],
  "obstacles": [],
  "emotions": [],
  "nextActions": []
}
```

MemoryItem

```
{
  "id": "mem_001",
  "type": "pattern",
  "content": "用户下午容易分心",
  "confidence": 0.78,
  "evidence": [],
  "lastUpdated": "2026-06-28"
}
```

Skill

```
{
  "id": "skill_daily_reflection",
  "name": "每日复盘诀",
  "type": "reflection",
  "score": 0.82,
  "params": {
    "tone": "supportive",
    "detailLevel": "medium"
  }
}
```

```
}
```

HarnessTrace

```
{  
  "id": "trace_001",  
  "input": "...",  
  "retrievedMemoryIds": [],  
  "selectedSkillIds": [],  
  "output": {},  
  "evaluation": {},  
  "memoryUpdates": [],  
  "skillUpdates": []  
}
```

11. MVP 范围

黑客松 MVP 必须收束成一个完整闭环。

必做

1. 修炼日志自然语言输入
2. Agent 自动生成今日复盘
3. Agent 识别心魔
4. Agent 生成明日修炼计划
5. Memory 更新展示
6. Skill 调用展示
7. Harness Trace 展示
8. 境界 / 子境界进度变化
9. 心魔雷达图
10. 命运长河时间线
11. 洞府主页视觉展示

可选

1. 真实 3D 洞府
2. Skill 自动生成
3. 复杂向量检索
4. 多用户系统
5. 日历集成
6. 移动端适配
7. 真实通知系统

暂不做

1. 完整社交系统
2. 完整游戏经济系统
3. 复杂任务管理器
4. 多 Agent 大规模协作
5. 全自动代码执行
6. 原作世界观完整复刻

12. Demo 剧本

现场 Demo 可以这样演：

Step 1：展示洞府主页

用户当前：

总境界：练气六层
AI Agent 子境界：练气八层
算法子境界：练气四层
当前心魔：下午分心、目标过大

Step 2：输入今日修炼日志

今天学了 RAG，但是下午刷短视频浪费了两小时。
算法题没做完，有点焦虑。
晚上看了 Agent Harness 的资料，感觉 LifeOS 项目思路清楚了一些。

Step 3：Agent 生成复盘

输出：

- 今日修炼成果
- 今日心魔
- 情绪状态
- 明日计划
- 项目推进建议

Step 4：展示 Harness Trace

系统展示：

检索 Memory：
- 用户最近三次下午学习效率较低
- 用户当前目标是完成 LifeOS Agent MVP

调用 Skills：
- 每日复盘诀
- 心魔照见法
- 周天规划术

- 记忆凝练术

评估：

- 计划难度偏高
- 情绪支持需求上升
- LifeOS 项目推进正相关

Step 5：展示自进化

系统更新：

Memory 更新：

“用户下午更适合轻量整理任务，上午适合深度学习。”

Skill Evolution：

planning.intensity: 0.8 → 0.6

planning.taskGranularity: normal → small

Step 6：展示可视化变化

- 心魔雷达图更新
- AI Agent 子境界进度提升
- 算法子境界保持不变
- 命运长河新增今日节点
- 洞府灵气 / 光效变化

13. 48 小时开发计划

第 0-4 小时：确定体验与数据结构

- 定义页面结构
- 定义 mock 数据
- 定义境界体系
- 定义 HarnessTrace 数据格式
- 确定 Demo 输入样例

第 4-12 小时：前端主界面

- 洞府主页
- 修炼日志页
- 心魔雷达图
- 子境界进度
- 时间线长河

第 12-20 小时：Agent 流程

- 输入解析
- Memory mock retrieval
- Skill selection
- Reflection generation
- Plan generation
- Harness trace generation

第 20-28 小时：自进化闭环

- Memory update 展示
- Skill params update 展示
- 境界进度更新逻辑
- 心魔变化逻辑

第 28-36 小时：视觉增强

- 3D / 伪 3D 洞府
- 动效
- 境界突破动画
- 时间线动画
- 光效和粒子效果

第 36-42 小时：路演模式

- 一键加载 Demo 数据
- 一键执行完整 Agent 流程
- Harness 面板可读性优化
- 准备固定 Demo 剧本

第 42-48 小时：打磨

- 修 bug
- 优化文案
- 准备 README
- 准备 PPT
- 录制演示视频
- 预演答辩

14. 核心亮点

亮点 1：修仙世界观不是皮肤，而是成长模型

境界、子境界、心魔、功法、洞府、命运长河都对应真实产品机制。

亮点 2：Agent Harness 让自进化可观测

每次 Agent 为什么这样建议、用了哪些 Memory、调用了哪些 Skills、如何更新自身，都能被展示。

亮点 3：Memory + Skills 双循环

Memory 让 Agent 越来越懂用户，Skills 让 Agent 越来越会帮用户。

亮点 4：视觉体验强

用户不是在看一个表格型工具，而是在进入自己的成长洞府，看见修炼轨迹、心魔变化和境界突破。

亮点 5：情绪价值真实

LifeOS Agent 面向独自学习和成长的人，解决的是长期陪伴、持续反馈和自我理解的问题。

15. 路演金句

传统日程工具只记录任务，传统 AI 助手只回答问题。

LifeOS Agent 试图成为一个长期陪伴用户成长的智能体。

它记录的不只是今天做了什么，而是一个人如何一步步修炼、受挫、调整、突破。

我们的 Agent Harness 不是日志系统，而是 Agent 自进化的训练场。

它记录每次建议、执行和反馈，并将这些轨迹转化为下一次更适合用户的计划。

在 LifeOS 里，功法就是 Skills，心魔就是阻碍模式，境界就是长期成长轨迹。

修仙不是皮肤，而是一套可解释、可视化、可进化的个人成长模型。

16. 最小成功标准

黑客松现场只要能稳定演示下面这条链路，就算 MVP 成功：

输入一段今日记录

→ Agent 解析修炼日志

→ 检索长期 Memory

→ 选择功法 Skills

→ 生成复盘和明日计划

→ Harness 展示决策链路

→ 更新 Memory 和 Skill 参数

→ 境界、心魔雷达图、命运长河、洞府主页同步变化

这条链路同时覆盖了：

- 产品体验
- 修仙世界观
- Memory
- Skills
- Agent Harness
- 自进化

- 可视化

它就是 LifeOS Agent 的核心闭环。